PROGRAMAÇÃO I

ESTRUTURAS DE DADOS EM PYTHON

VOLUME II • CADERNO 06

COMPLEXIDADE, ÁRVORES E GRAFOS

Como avaliar o crescimento dos algoritmos e reconhecer estruturas com hierarquias e conexões.

COMO ESTUDAR Comece pelo problema concreto, acompanhe o estado dos dados antes e depois de cada operação e só então leia o código. Execute os exemplos na ordem. Quando aparecer um termo técnico, ele será definido no próprio capítulo. Não é necessário decorar comandos antes de entender o comportamento.

CONTEXTO DIDÁTICO Os exemplos acompanham uma rede fictícia de lojas, com produtos, clientes, estoque, pedidos e atendimento. Os registros são exemplos de aula, não dados comerciais reais. Este caderno pressupõe os fundamentos de Python e o Caderno 01 (listas).

Série de cadernos independentes e complementares • Texto explicativo, exemplos executáveis e roteiros físicos de representação.

Mapa de aprendizagem deste caderno

Este caderno conclui a trilha principal com análise intuitiva de eficiência e oferece uma extensão introdutória de estruturas hierárquicas e conexões. O aluno compara custos por número de operações antes de encontrar a notação formal e relaciona os novos conceitos às estruturas já estudadas.

As representações em tabelas mostram o que o computador armazena; as demonstrações físicas mostram como uma pessoa poderia simular as mesmas operações com cartões ou colegas em posições fixas. **O número da posição não se move: o elemento é que muda.**

1. O que significa um algoritmo ser eficiente?

Imagine uma loja com dez produtos e outra com dez mil. Dois procedimentos podem produzir o mesmo resultado, mas ter comportamentos distintos quando a quantidade de produtos aumenta. A análise de eficiência estuda **como cresce o trabalho necessário** em função do tamanho da entrada. Chamaremos esse tamanho de **n**.

O tempo exato em segundos depende da máquina, linguagem, ambiente e tamanho real dos dados. Para compreender algoritmos, costuma ser mais útil observar quantas operações relevantes são realizadas conforme n aumenta. A notação **Big O** descreve uma ordem de crescimento superior assintótica, e é frequentemente usada para expressar o comportamento de pior caso.

Quantidade n	Busca linear: até n verificações	Seleção: n(n−1)/2 comparações
5	5	10
10	10	45
20	20	190
100	100	4950

Quando passamos de 10 para 20 elementos, o pior caso da busca linear dobra de 10 para 20 verificações. Já a ordenação por seleção passa de 45 para 190 comparações — mais de quatro vezes. A diferença não vem de um computador mais lento, mas da organização do procedimento.

2. Complexidade constante: O(1)

Uma operação é considerada constante quando seu custo não cresce com a quantidade total de elementos, dentro do modelo analisado. Consultar um índice válido de uma lista Python é exemplo típico: a instrução usa uma posição já conhecida, sem precisar examinar os elementos anteriores.

```
precos = [24.90, 18.50, 59.90, 14.90]
print(precos[2])  # 59.9
```

O acesso ao índice 2 não exige comparar com índices 0 e 1. O comportamento é diferente de procurar um preço desconhecido: nesse caso, precisamos descobrir a posição. A notação não significa que a operação leva exatamente um segundo ou um ciclo de processamento; significa apenas que seu crescimento não depende de n de forma relevante.

3. Complexidade linear: O(n)

Percorrer uma lista inteira visita cada elemento uma vez. Se a lista possui n elementos, o número de visitas cresce proporcionalmente a n. A busca linear também tem pior caso O(n), pois pode ter de examinar todos os elementos antes de encontrar ou descartar o alvo.

```
def contar_estoque(quantidades):
```

```
    total = 0
    for qtd in quantidades:
        total += qtd
    return total
```

n	Operações principais do laço
4	4
8	8
16	16
32	32

Mesmo que cada repetição envolva mais de uma instrução, o crescimento continua linear quando o custo por elemento é aproximadamente constante. É por isso que Big O ignora multiplicadores fixos ao comparar comportamentos assintóticos: O(3n) é classificado como O(n).

4. Complexidade quadrática: O(n²)

Um laço que percorre todos os elementos dentro de outro laço que também percorre todos normalmente realiza uma quantidade de pares proporcional a n^2. Alguns algoritmos de ordenação por comparação executam menos operações específicas, como $n(n-1)/2$, mas a ordem de crescimento ainda é quadrática.

```
codigos = [101, 102, 103, 104]
for primeiro in codigos:
    for segundo in codigos:
        print(primeiro, segundo)
```

A saída tem 16 pares quando n = 4, pois cada um dos quatro valores é combinado com outros quatro. Se n = 10, serão 100 pares. É diferente de dois laços executados um depois do outro: duas passagens completas somam aproximadamente 2n operações e continuam O(n).

n	n	n^2	$n \log_2 n$ (aprox.)
8	8	64	24
16	16	256	64
32	32	1024	160
64	64	4096	384

5. Complexidade logarítmica: O(log n)

Quando uma operação elimina metade das possibilidades a cada passo, o tamanho do problema diminui rapidamente. A busca binária é o exemplo clássico. Com 16 posições, podemos reduzir a região possível para 8, depois 4, depois 2 e finalmente 1.

Quantidade inicial	Metades possíveis até restar 1	Reduções
8	8 → 4 → 2 → 1	3
16	16 → 8 → 4 → 2 → 1	4
32	32 → 16 → 8 → 4 → 2 → 1	5
64	64 → 32 → 16 → 8 → 4 → 2 → 1	6

O logaritmo de base 2 responde aproximadamente quantas vezes podemos dividir um número por dois até chegar a 1. O número exato de comparações em uma busca binária varia conforme a

entrada e o alvo; a tabela apresenta reduções sucessivas para ilustrar a ordem de crescimento, e não uma contagem universal de comparações.

6. O(n log n): dividir e ainda processar os elementos

Merge Sort divide os dados em subproblemas menores e realiza uma intercalação que examina os elementos. Como existem cerca de $\log_2(n)$ níveis de divisão e, em cada nível, o trabalho total é proporcional a n, o resultado é O(n log n).

n	Níveis de divisão (aprox.)	$n \times \log_2 n$
8	3	24
16	4	64
32	5	160
64	6	384

Quick Sort também pode apresentar O(n log n) quando as partições são razoavelmente equilibradas. Entretanto, pivôs desfavoráveis podem produzir regiões muito desiguais e levar a $O(n^2)$. Por isso, ao comparar algoritmos, é útil distinguir **melhor caso, caso médio e pior caso**, em vez de citar um único número como verdade universal.

7. Analisar tempo e espaço separadamente

Um algoritmo pode fazer poucas comparações e, ao mesmo tempo, reservar novas listas temporárias. A análise de **espaço auxiliar** observa o crescimento da memória adicional, sem contar necessariamente a memória já ocupada pela entrada original. Esse critério importa quando o volume de dados é grande.

Algoritmo / operação	Tempo típico ou pior caso	Espaço auxiliar (versões deste material)
Acesso lista[i]	O(1)	O(1)
Busca linear	O(n) pior caso	O(1)
Busca binária iterativa	O(log n) pior caso	O(1)
Selection Sort (cópia didática)	$O(n^2)$	O(n) pela cópia devolvida
Merge Sort (slicing + intercalação)	O(n log n)	O(n) auxiliar de listas; pilha O(log n)
Quick Sort didático com três listas	O(n log n) médio; $O(n^2)$ pior	Aloca listas temporárias; varia com a partição

A tabela é intencionalmente específica: nossos exemplos de ordenação devolvem uma cópia, mesmo quando o algoritmo clássico poderia operar in-place. Não faria sentido atribuir espaço O(1) à função do Caderno 04 que começa com `dados = valores.copy()`.

8. Recursão e o uso da pilha de chamadas

Na recursão, cada chamada de função aguarda que uma chamada menor termine. Isso pode ser representado por uma pilha: a chamada mais recente fica no topo e precisa terminar antes de devolver o controle à anterior. Esse mecanismo explica por que a recursão consome memória adicional.

```
def fatorial(n):
    if n == 0:
        return 1
    return n * fatorial(n - 1)
```

```
print(fatorial(4))  # 24
```

Chamada	Espera o resultado de
fatorial(4)	4 × fatorial(3)
fatorial(3)	3 × fatorial(2)
fatorial(2)	2 × fatorial(1)
fatorial(1)	1 × fatorial(0)
fatorial(0)	retorna 1; começa a desempilhar

O caso-base é `n == 0`; sem ele, a função não terminaria corretamente. A implementação supõe n inteiro não negativo. Para muitos cálculos simples, uma solução iterativa é mais eficiente em memória do que construir uma cadeia profunda de chamadas recursivas.

9. Árvores: representar hierarquias

Algumas informações têm uma organização hierárquica: uma categoria pode conter subcategorias e cada subcategoria pode conter produtos. Uma **árvore** é uma estrutura de nós conectados sem ciclos na representação hierárquica clássica. O primeiro nó é a **raiz**; nós abaixo são filhos; um nó sem filhos é chamado de folha.

Nó	Pai	Filhos
Produtos	— (raiz)	Higiene, Beleza
Higiene	Produtos	Sabonetes, Desodorantes
Beleza	Produtos	Maquiagem, Perfumaria
Sabonetes	Higiene	— (folha)

DINÂMICA Um aluno segura a placa “Produtos” e fica à frente. Dois colegas representam categorias e ficam atrás, ligados por barbantes ou setas desenhadas em papel. Outros representam subcategorias. O professor indica uma categoria; os alunos acompanham o caminho desde a raiz até a folha, nomeando cada ligação.

Em uma **árvore binária**, cada nó possui no máximo dois filhos. Uma árvore binária de busca acrescenta uma regra de ordenação relativa entre chaves: por exemplo, menores à esquerda e maiores à direita em uma convenção simples. Nem toda árvore binária é uma árvore binária de busca.

10. Uma árvore binária de busca didática

Considere inserir os valores 40, 20, 60, 10, 30 e 50 em uma árvore de busca que coloca valores menores à esquerda e maiores à direita. O valor 40 será a raiz. Em seguida, 20 fica à esquerda e 60 à direita; 10 e 30 serão filhos de 20; 50 será filho esquerdo de 60.

```
arvore = {
    "valor": 40,
    "esquerda": {
        "valor": 20,
        "esquerda": {"valor": 10, "esquerda": None, "direita": None},
        "direita": {"valor": 30, "esquerda": None, "direita": None}
    },
    "direita": {
        "valor": 60,
        "esquerda": {"valor": 50, "esquerda": None, "direita": None},
        "direita": None
    }
```

```
}
```

A representação em dicionários facilita visualizar cada nó como um valor e duas referências. Uma busca começa na raiz e segue para o filho esquerdo ou direito conforme a comparação. Se a árvore estiver muito desequilibrada, porém, pode se comportar como uma lista encadeada em termos de quantidade de passos.

11. Grafos: representar conexões

Uma árvore é adequada a uma hierarquia, mas nem todas as relações são hierárquicas. Lojas podem estar ligadas por várias rotas; cada cidade pode ter caminhos para outras cidades. Um **grafo** descreve entidades chamadas **vértices** e conexões chamadas **arestas**.

Origem	Destino	Distância didática
Sete Lagoas	Contagem	65 km
Contagem	Belo Horizonte	18 km
Sete Lagoas	Belo Horizonte	72 km
Belo Horizonte	Betim	32 km

As distâncias acima são números ilustrativos de aula, não estimativas rodoviárias oficiais. Para representar um grafo pequeno, um dicionário pode associar cada vértice à lista de seus vizinhos. Essa representação é conhecida como **lista de adjacência**.

```
rotas = {
    "Sete Lagoas": ["Contagem", "Belo Horizonte"],
    "Contagem": ["Sete Lagoas", "Belo Horizonte"],
    "Belo Horizonte": ["Sete Lagoas", "Contagem", "Betim"],
    "Betim": ["Belo Horizonte"]
}
print(rotas["Contagem"])
```

REPRESENTAÇÃO FÍSICA Quatro estudantes representam cidades. Fitas no chão representam conexões. É possível chegar a uma cidade por diferentes caminhos, ao contrário de uma hierarquia simples de categorias. O trajeto escolhido precisa respeitar as arestas realmente existentes.

12. Busca em largura em grafos: uma fila em ação

A **busca em largura (BFS)** explora primeiro os vizinhos diretos de um vértice e, em seguida, os vizinhos desses vizinhos. Uma fila ajuda a manter a ordem de exploração. Em grafos sem pesos, ou quando cada aresta tem o mesmo custo, a BFS encontra caminhos com o menor número de arestas.

```
from collections import deque

def bfs(grafo, inicio):
    visitados = {inicio}
    fila = deque([inicio])
    ordem = []
    while fila:
        atual = fila.popleft()
        ordem.append(atual)
        for vizinho in grafo.get(atual, []):
            if vizinho not in visitados:
                visitados.add(vizinho)
                fila.append(vizinho)
    return ordem

print(bfs(rotas, "Sete Lagoas"))
```

O conjunto `visitados` impede que um vértice seja colocado repetidamente na fila quando há ciclos no grafo. A ordem exata de visita depende da ordem dos vizinhos. Em grafos com distâncias diferentes entre cidades, BFS **não garante o caminho de menor quilometragem**; um algoritmo apropriado para pesos não negativos, como Dijkstra, trata outro problema.

13. Síntese: estruturas e algoritmos são escolhas de projeto

Problema	Estrutura / algoritmo inicial	Raciocínio
Localizar registro por chave única	dict / tabela hash	A chave identifica o valor.
Manter histórico de desfazer	pilha	Sai o registro mais recente.
Atender em ordem de chegada	fila / deque	Sai a senha mais antiga.
Encontrar código em dados ordenados	busca binária	Descarta metades.
Ordenar grandes coleções	sorted() ou algoritmo O(n log n)	Crescimento importa.
Representar categorias e subcategorias	árvore	Existe relação hierárquica.
Representar ligações entre lojas	grafo	Existem múltiplos caminhos possíveis.

O estudo termina com um princípio simples: **primeiro identifique as operações necessárias; depois escolha a estrutura e o algoritmo.** O mesmo problema pode admitir várias soluções corretas, mas elas não terão necessariamente a mesma clareza, custo e facilidade de manutenção.

Para guardar deste caderno

- Big O descreve a evolução do trabalho, não o tempo exato em segundos.
- Operações $O(1)$, $O(n)$, $O(\log n)$, $O(n \log n)$ e $O(n^2)$ têm ritmos de crescimento diferentes.
- A análise de espaço deve considerar as cópias realmente criadas pela implementação.
- Árvores representam hierarquias; grafos representam conexões mais gerais.
- BFS usa fila e conjunto de visitados e minimiza número de arestas em grafos sem pesos.

Referências de consulta

Python Software Foundation. The Python Tutorial: Data Structures. https://docs.python.org/3/tutorial/datastructures.html

Python Software Foundation. Biblioteca padrão: collections e json. https://docs.python.org/3/library/collections.html ; https://docs.python.org/3/library/json.html

VisuAlgo. Visualizações e explicações de algoritmos. https://visualgo.net/en

Harvard University. CS50x 2026, Week 3 — Algorithms. https://cs50.harvard.edu/x/weeks/3/

Bhargava, Aditya. Entendendo Algoritmos (Grokking Algorithms), 2ª edição. Manning Publications. Referência de abordagem visual; exemplos e explicações deste caderno são originais.